

Topic 02: Time Travel & Parallel Realities: Git Internals

Basic Software Engineering Workflows

TOPIC 02

Time Travel & Parallel Realities: Git Internals

Directed Acyclic Graphs, SHA-1 snapshots, branching, and merge conflict surgery.

Cognitive Objectives

By the end of this topic, learners will be able to:

- **Model Git internals** as a content-addressable directed acyclic graph (DAG) of Blobs, Trees, and Commits.
- **Deconstruct branch pointers** as lightweight 41-byte SHA-1 text files rather than heavy data copies.
- **Choose between 3-Way Merging and Rebasing** to maintain clear, auditable collaborative commit histories.
- **Resolve merge conflicts defensively** without losing critical upstream data science code or model weights.

The Mental Model: Diffs vs Snapshots

The Intuitive Misconception: Delta Diffs

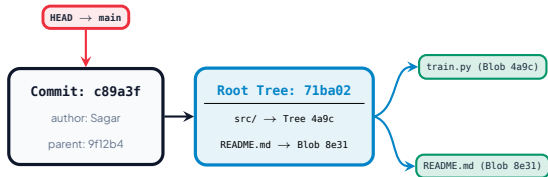
- Common belief: Git stores a base file plus a series of incremental text diffs (+10 lines, -3 lines).
- **Flaw:** Reconstructing a file from 10,000 commits would require slow sequential diff playback.
- How does `git checkout` switch across branches in 2 milliseconds?

The "Aha!" Discovery: Directed Acyclic Graph

- **Git stores snapshots, not diffs!**
- Every commit points to a complete root **Tree** snapshot of the entire repository.
- Unchanged files share identical **Blob** pointers via content-hash deduplication (SHA-1 / SHA-256).
- Branches are merely 41-byte text pointers moving along the commit DAG!

PUNCHLINE Git is a content-addressable key-value object database.

Hero Visualization: The Git Object Model (Blobs, Trees, Commits)

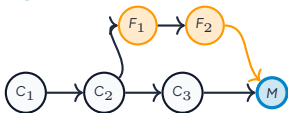


Content-Addressed Storage

- **Blob:** Raw binary payload of file contents. File names and permissions are stored in Trees, not Blobs.
- **Tree:** Directory mapping holding file names, permissions, and hash pointers to child Trees / Blobs.
- **Commit:** Author metadata, timestamp, commit message, and pointer to parent commit hash.

Hero Visualization: 3-Way Merge vs Rebase Topologies

3-Way Merge: Preserves Branch Divergence



Topology: Non-linear history. Retains exact chronological branching timestamps and diamond merge commit M .

Rebase: Clean Linear Narrative



Topology: Completely linear DAG. Replays feature commits on top of the latest **main**. Eliminates messy diamond merge clutter.

Production Checkpoint: Reset vs Revert Safety Protocol

Local Sandbox: `git reset`

- **Action:** Rewinds branch ref to an earlier commit and discards unpushed commits.
- **Use Case:** Local feature branch before pushing upstream.
- **Golden Rule:** **Never** reset commits that have already been pushed to shared remote branches!

Shared Remote: `git revert`

- **Action:** Creates a new forward-moving commit that applies the exact inverse patch of the target commit.
- **Use Case:** Rolling back broken production deployments without rewriting shared history.
- Preserves DAG integrity for all team collaborators.

Use `git revert` for shared
branches to protect
collaboration.